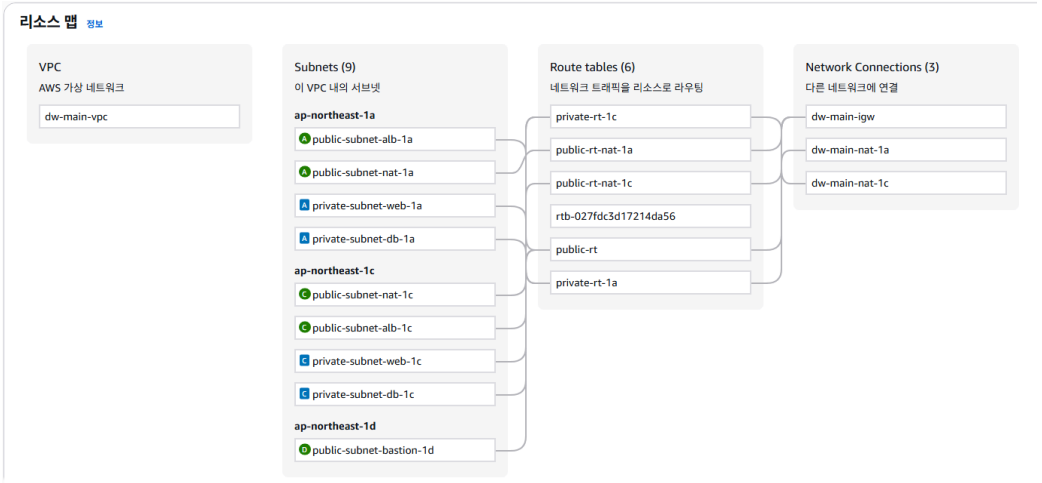
3주차 공통 실습

업무 영역	Devops 자동화
진행현황	작업완료
담당자	동욱 김
진행률	1
설명	[O] ALB + ASG 구성 [O] 모니터링 & 로그 확인 [O] CI/CD & 배포 테스트
주차	3주차

▼ ALB + ASG 구성

1. vpc 구성



- 프로젝트 아키텍처와 유사한 vpc 구성

2. 보안 그룹 구성

보안 그룹 이름	인바운드	아웃바운드
WEB-SG	http, https → ALB-SG SSH → BASTION-SG	http,https → 0.0.0.0/0 DNS(TCP, UDP) → 10.0.0.2/32
BASTION-SG	SSH → 내 IP	http, https → 0.0.0.0/0 SSH → WEB-SG

		DNS (UDP) → 10.0.0.2/32 DNS (TCP) → 10.0.0.23/32
ALB-SG	http, https → 0.0.0.0/0	http, https → WEB-SG

3. 시작 템플릿 구성

구성	값
이름	web-lt
보안 그룹	WEB-SG

```
#!/bin/bash
set -euxo pipefail
export DEBIAN_FRONTEND=noninteractive

# 1) 기본 패키지
apt-get update -y
apt-get install -y python3-pip python3-venv nginx curl

# 2) 앱 디렉토리/코드
APP_DIR=/opt/flaskapp
mkdir -p "$APP_DIR"
cat > "$APP_DIR/app.py" <<'PY'
from flask import Flask
app = Flask(__name__)

@app.get("/health")
def health():
    return "OK", 200

@app.get("/")
def home():
    return "Hello, ALB!", 200
PY

# 3) 가상환경 + 패키지
python3 -m venv "$APP_DIR/venv"
"$APP_DIR/venv/bin/pip" install --upgrade pip
"$APP_DIR/venv/bin/pip" install flask gunicorn
```

```
# 퍼미션(ubuntu 사용자로 실행)
chown -R ubuntu:www-data "$APP_DIR"

# 4) systemd(gunicorn)
cat > /etc/systemd/system/flask-gunicorn.service <<'SERVICE'
[Unit]
Description=Gunicorn serving Flask app
After=network.target

[Service]
User=ubuntu
Group=www-data
WorkingDirectory=/opt/flaskapp
Environment=PATH=/opt/flaskapp/venv/bin
ExecStart=/opt/flaskapp/venv/bin/gunicorn -w 2 -b 127.0.0.1:8000 app:a
pp
Restart=always

[Install]
WantedBy=multi-user.target
SERVICE

systemctl daemon-reload
systemctl enable --now flask-gunicorn

# 5) Nginx(80 → 127.0.0.1:8000)
cat > /etc/nginx/sites-available/flaskapp <<'NGINX'
server {
    listen 80 default_server;
    server_name _;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
```

```
    location /health {
        proxy_pass http://127.0.0.1:8000/health;
    }
}
NGINX

rm -f /etc/nginx/sites-enabled/default
ln -sf /etc/nginx/sites-available/flaskapp /etc/nginx/sites-enabled/flaska
pp
nginx -t
systemctl enable --now nginx

# 6) 부팅 중 간단 자기진단
sleep 2
curl -sI http://127.0.0.1/health | head -n1 || true
```

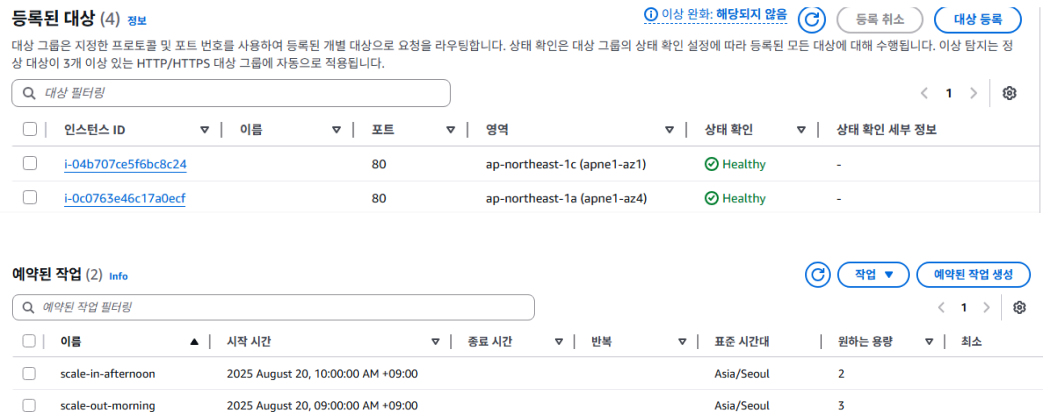
4. 타겟 그룹 구성



5. alb 구성
    a. 로드밸런서 - 로드밸런서 생성 클릭
    b. Application Load Balancer 생성 클릭
    c. vpc를 1에서 구성한 vpc로 선택
    d. 가용 영역을 1a, 1c로 설정하고 alb-1a, alb-1c 서브넷을 선택한다.
    e. 보안 그룹 ALB-SG를 선택한다
    f. 리스너 및 라우팅에서 포트 80, 기본 작업을 4번에서 구성한 타겟 그룹을 선택한다.
6. asg 구성
    a. 이름을 설정하고, 시작 템플릿을 3번에서 구성한 것으로 선택한다.
    b. vpc, 가용 영역, 서브넷을 설정한다.

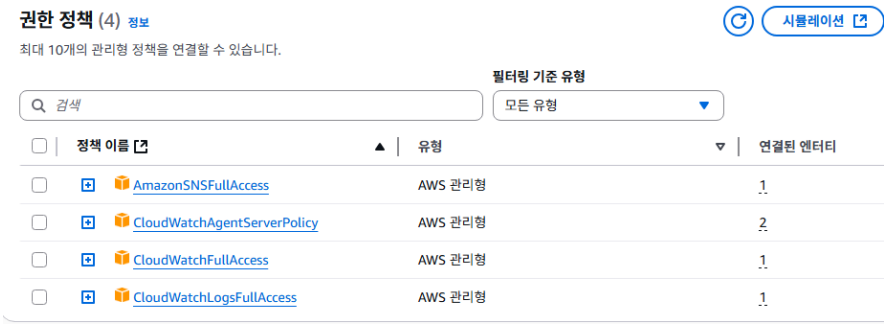
c. 로드 밸런서를 설정한다.

d. 인스턴스 크기, 최소, 최대를 설정한다.



▼ 모니터링 & 로그 확인

1. IAM 역할 설정



2. 임시로 퍼블릭 ec2 생성 후 putty 접속

3. CloudWatch Agent 설치 (Ubuntu)

```
curl -fsSL https://s3.amazonaws.com/amazoncloudwatch-agent/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb -o /tmp/cw.deb
sudo dpkg -i /tmp/cw.deb
```

4. 메타데이터 접근을 위한 IMDSv2 토큰 발급

```
# 토큰 발급 (유효시간 6시간)
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

# 토큰이 비어있으면 메타데이터 비활성화/차단 가능성
```

```
[ -z "$TOKEN" ] && echo "IMDS 토큰 발급 실패: 인스턴스 메타데이터가 비활성/차단일 수 있음" || echo "IMDSv2 토큰 OK"

# EC2에 붙은 IAM Role 이름 조회
ROLE=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/iam/security-credentials/)
echo "ROLE=$ROLE"
```

```
# 역할/권한 OK 후, 에이전트 적용
sudo systemctl restart amazon-cloudwatch-agent
sudo systemctl status amazon-cloudwatch-agent --no-pager
```

5. 알람을 받을 이메일 설정

```
# 필요 패키지
sudo apt update -y && sudo apt install -y unzip curl

# 아키텍처 감지 후 다운로드
ARCH=$(uname -m)
URL="https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip"
[ "$ARCH" = "aarch64" ] || [ "$ARCH" = "arm64" ] && \
  URL="https://awscli.amazonaws.com/awscli-exe-linux-aarch64.zip"

cd /tmp
curl -fsSL "$URL" -o awscliv2.zip
unzip -q awscliv2.zip

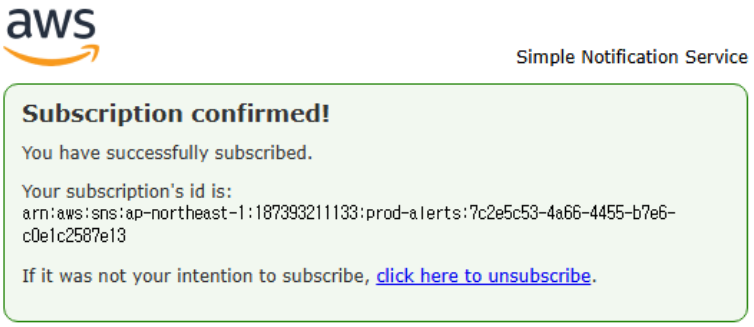
# 설치(기존 설치 있으면 --update)
sudo ./aws/install --update

# 확인
aws --version
which aws
```

```
# SNS 토픽 생성
aws sns create-topic --name prod-alerts --region ap-northeast-2
```

```
# 이메일 구독 (이메일 주소 교체)
# <ACCOUNT_ID> = AWS 콘솔 우측 상단 12자리 숫자
# you@example.com = 자신의 이메일
aws sns subscribe \
  --topic-arn arn:aws:sns:ap-northeast-1:<ACCOUNT_ID>:prod-alert
s \
  --protocol email \
  --notification-endpoint you@example.com \
  --region ap-northeast-1
```

- 받은 메일에서 "Confirm subscription" 반드시 클릭



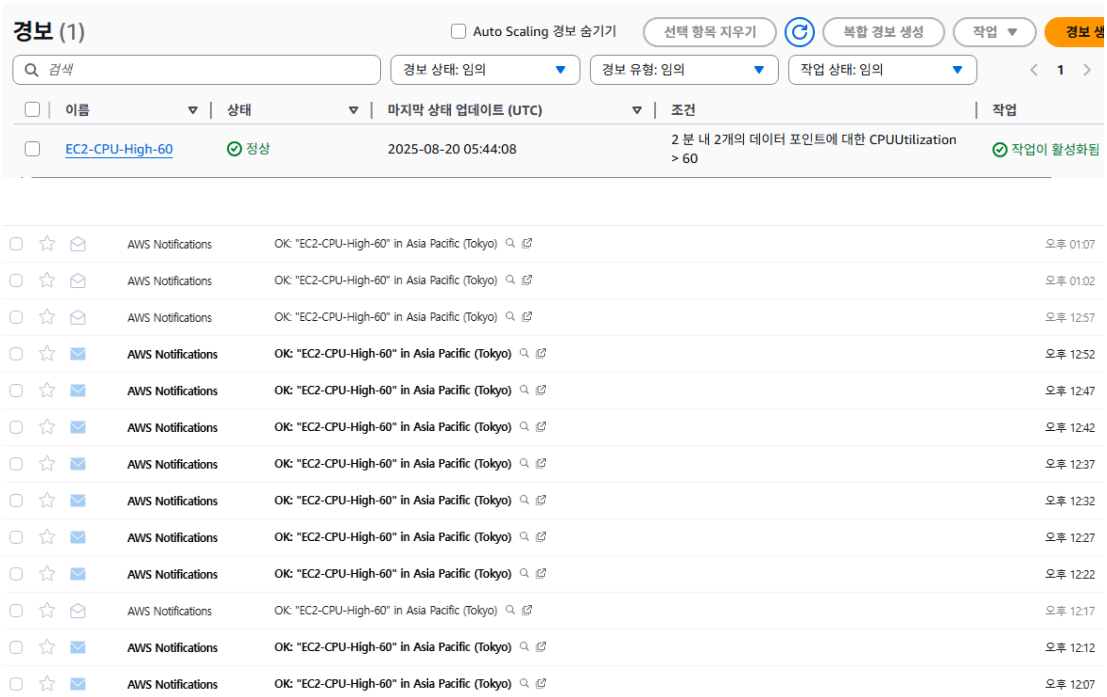
6. 알람 설정

```
# TOPIC_ARN 확인
TOPIC_ARN=$(aws sns create-topic --name prod-alerts --query To
picArn --output text)
[ -z "$TOPIC_ARN" ] && echo "CreateTopic 실패(권한/리전 확인)" && e
xit 1
echo "Topic: $TOPIC_ARN"

# <InstanceId>: AWS 콘솔에서 인스턴스 ID 확인
# <TOPIC_ARN>: 위 코드에서 확인
aws cloudwatch put-metric-alarm \
  --alarm-name "EC2-CPU-High-60" \
  --metric-name CPUUtilization \
  --namespace AWS/EC2 \
  --statistic Average \
```

```
--period 300 \
--evaluation-periods 1 \
--threshold 60 \
--comparison-operator GreaterThanThreshold \
--dimensions Name=InstanceId,Value=<InstanceId>\
--alarm-actions <TOPIC_ARN> \
--region ap-northeast-1
```

7. CloudWatch Logs 및 이메일 확인

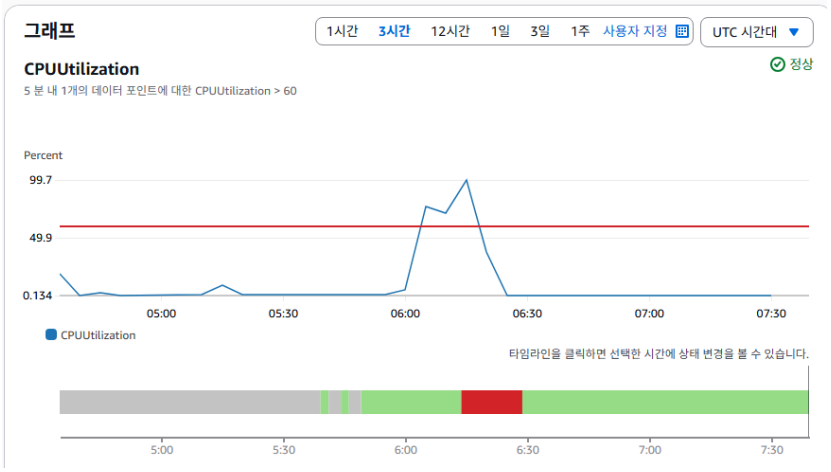


8. CPU 부하 테스트

- CPU 사용량을 강제로 60퍼 이상으로 전환

```
V=$(nproc)
P=$(( (V * 60 + 99) / 100 ))   # 60%에 해당하는 프로세스 수
for i in $(seq 1 $P); do yes > /dev/null & done
```

- CloudWatch Logs 및 이메일 확인



영어에서 한국어로 번역되었습니다. 원문보기

이 이메일을 받는 이유는 아시아 태평양(도쿄) 지역의 Amazon CloudWatch 알람 "EC2-CPU-High-60"이 알람 상태에 진입했기 때문입니다. 이는 "교차 임계값: 1 데이터 포인트 [99.70967741935482 (20/08/25 08:28:00)]가 임계값(60.0)보다 컸기 때문입니다. "2025년 8월 20일 수요일 08:33:44 UTC"에서.

AWS 관리 콘솔에서 이 알람 보기:
https://ap-northeast-1.console.aws.amazon.com/cloudwatch/deeplink.js?region=ap-northeast-1#alarmsV2:alarm/EC2-CPU-High-60

알람 세부 정보:
- 이름: EC2-CPU-하이-60
- 설명:
- 상태 변경: 확인 -> 알람
- 상태 변경 이유: 임계값 초과: 1 데이터 포인트 [99.70967741935482 (20/08/25 08:28:00)]가 임계값(60.0)보다 높았습니다.
- 타임스탬프: 2025년 8월 20일 수요일 08:33:44 UTC
- AWS 계정: 236528210774
- 알람 Arn: arn:aws:cloudwatch:ap-northern-1:236528210774:alarm:EC2-CPU-High-60

임계값:
- 메트릭이 300초의 마지막 1주기 중 최소 1주기 동안 GreaterThanThreshold 60.0일 때 알람은 ALARM 상태입니다.

9. `/project/nginx/access` 로그 그룹 생성

```
# 로그 그룹 생성(이미 있으면 에러 나와도 무시해도 됨)
aws logs create-log-group --log-group-name /project/nginx/access || true

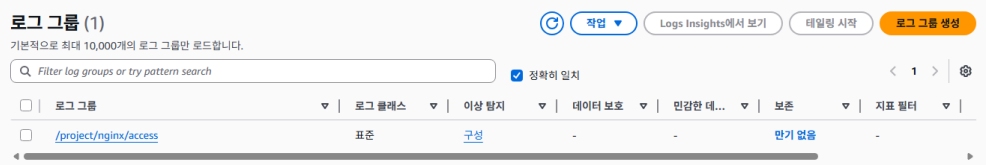
# jq 없으면 설치
sudo apt update -y && sudo apt install -y jq

# logs 섹션 추가
sudo tee /opt/aws/amazon-cloudwatch-agent/etc/logs.d/project-nginx-access.json >/dev/null <<'JSON'
{
  "logs": { "logs_collected": { "files": { "collect_list": [
    {
      "file_path": "/var/log/nginx/access.log",
      "log_group_name": "/project/nginx/access",
```

```
      "log_stream_name": "{instance_id}",
      "timestamp_format": "%d/%b/%Y:%H:%M:%S %z"
    }
  ]}}}
}
JSON

# 적용
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
  -a fetch-config -m ec2 \
  -c file:/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json \
  -s
```

- 이후 AWS 콘솔 - cloudwatch - 로그 그룹에서 확인



▼ CI/CD & 배포 테스트

aws 실습 구성-c (1)

aws 실습 구성-d (1)

- 실습에 따라 배포 테스트 진행
- 일부 코드 개선

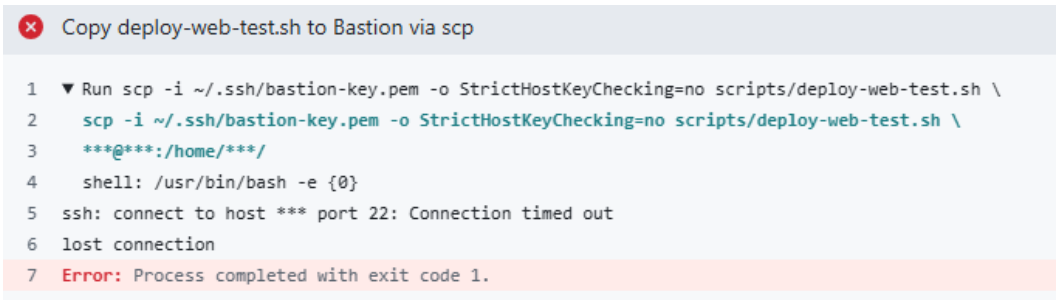
1. Dockerfile 경로가 현재 디렉토리로 설정. 프로젝트의 설정 경로에 맞춰 변경

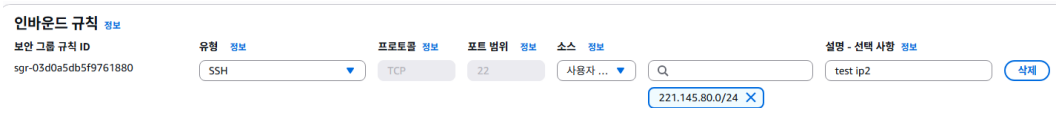
```
# 이전 코드
#- name: Build Docker image with Git SHA as APP_VERSION
#  run: |
#    docker build \
```

```
#      --build-arg APP_VERSION=${{ github.sha }} \
#      -t ${{ secrets.DOCKER_USERNAME }}/flask-app:latest .

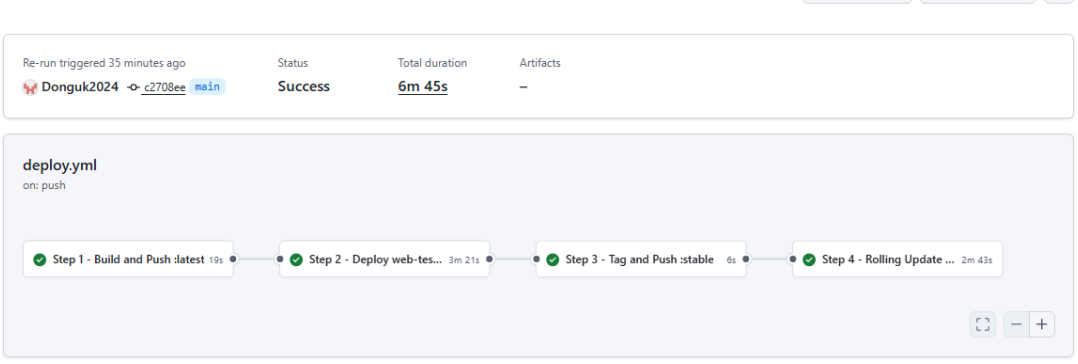
#절대 경로를 설정한 코드
 - name: Build Docker image with Git SHA as APP_VERSION
   run: |
     docker build \
       -f flask-app/Dockerfile \
       --build-arg APP_VERSION=${{ github.sha }} \
       -t ${{ secrets.DOCKER_USERNAME }}/actions_test:latest \
       flask-app
```

2. Bastion-SG의 인바운드 규칙에 ubuntu-runner의 ip가 없으면 연결 불가. 보안 그룹 규칙 추가





- 결과 확인



▼ 다음에 할 일

- github secrets 설정
- app.py 연동